

## Guía Pedagógica: Día 12 - Skeleton Loaders (Experiencia de Usuario)

Hoy vamos a llevar la **Experiencia de Usuario (UX)** de nuestra aplicación al siguiente nivel. En lugar de mostrar un aburrido ícono girando (spinner) o dejar la pantalla en blanco mientras esperamos los datos de la API, implementaremos **Skeleton Loaders** (Esqueletos de Carga).

Estos "esqueletos" simulan la forma que tendrá el contenido (como las tarjetas de películas) y tienen una animación de brillo (shimmer) que indica que el sistema está trabajando.

### Contexto Pedagógico: ¿Por qué Skeletons y no Spinners?

1. **Ilusión de Velocidad:** Psicológicamente, los usuarios sienten que la página carga más rápido si ven inmediatamente la estructura de la página en lugar de una pantalla vacía.
2. **Evitan el Layout Shift (CLS):** Si el espacio no está reservado, cuando la imagen carga de golpe empuja todo el contenido hacia abajo, rompiendo la lectura del usuario. Los skeletons reservan el tamaño exacto, manteniendo la estabilidad visual (una métrica vital para Google y el SEO).

### Paso 1: Estilos Globales de la Animación "Shimmer"

La magia de un Skeleton es su animación de brillo continuo. Para no repetir código, la definiremos en nuestros estilos globales para que cualquier componente pueda usarla.

Abre el archivo de estilos globales **src/styles.css** (o el equivalente donde tengas tus variables) y añade esto al final:

```

/* =====
ANIMACIONES GLOBALES (UX)
===== */

/* Animación de brillo para los Skeleton Loaders */
@keyframes shimmer {
  0% {
    background-position: -1000px 0;
  }
  100% {
    background-position: 1000px 0;
  }
}

/* Clase de utilidad reutilizable */
.skeleton-base {
  background: #2a2a2a;
  background-image: linear-gradient(
    to right,
    #2a2a2a 0%,
    #3a3a3a 20%,
    #2a2a2a 40%,
    #2a2a2a 100%
  );
  background-repeat: no-repeat;
  background-size: 1000px 100%;
  animation: shimmer 2s infinite linear forwards;
}

```

## Paso 2: Crear el Componente SkeletonCard

Este componente imitará la forma exacta de nuestro MovieCard (mismo ancho y alto).

1. Crea la carpeta: `src/app/shared/components/skeleton-card/`
2. Crea el archivo TypeScript `skeleton-card.ts`:

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-skeleton-card',
  standalone: true,
  template: `
    <div class="skeleton-card">
      <div class="skeleton-poster skeleton-base"></div>
      <div class="skeleton-info">
        <div class="skeleton-title skeleton-base"></div>
        <div class="skeleton-date skeleton-base"></div>
      </div>
    </div>
  `,
  styleUrls: ['./skeleton-card.css']
})
export class SkeletonCard {}

```

### 3. Crea el archivo CSS skeleton-card.css:

```
.skeleton-card {
  width: 100%;
  border-radius: 12px;
  overflow: hidden;
  background-color: #1a1a1a;
  display: flex;
  flex-direction: column;
  gap: 10px;
  padding-bottom: 10px;
}

.skeleton-poster {
  width: 100%;
  aspect-ratio: 2 / 3; /* Exactamente la misma proporción que el póster real */
  border-radius: 12px;
}

.skeleton-info {
  padding: 0 10px;
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.skeleton-title {
  width: 80%;
  height: 20px;
  border-radius: 4px;
}

.skeleton-date {
  width: 50%;
  height: 14px;
  border-radius: 4px;
}
```

### Paso 3: Crear el Componente SkeletonHero

Ahora crearemos el esqueleto para la sección principal (la película gigante que sale al principio en la Home).

1. Crea la carpeta: src/app/shared/components/skeleton-hero/
2. Crea el archivo skeleton-hero.ts:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-skeleton-hero',
  standalone: true,
  template: `
    <div class="skeleton-hero-wrapper skeleton-base">
      <div class="skeleton-hero-content">
        <div class="skeleton-badge skeleton-base"></div>
        <div class="skeleton-title skeleton-base"></div>
        <div class="skeleton-title short skeleton-base"></div>
        <div class="skeleton-overview skeleton-base"></div>
        <div class="skeleton-overview short skeleton-base"></div>
        <div class="skeleton-button skeleton-base"></div>
      </div>
    </div>
  `,
  styleUrls: ['./skeleton-hero.css']
})
export class SkeletonHero {}
```

### 3. Crea el archivo skeleton-hero.css:

```
.skeleton-hero-wrapper {
  width: 100%;
  height: 70vh;
  min-height: 500px;
  position: relative;
  /* Sobrescribimos el fondo para que el Hero sea más oscuro */
  background: #111;
}

.skeleton-hero-content {
  position: absolute;
  bottom: 10%;
  left: 5%;
  width: 90%;
  max-width: 600px;
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.skeleton-badge {
  width: 100px; height: 25px; border-radius: 4px;
}

.skeleton-title {
  width: 90%; height: 50px; border-radius: 8px;
}
.skeleton-title.short { width: 60%; }

.skeleton-overview {
  width: 100%; height: 18px; border-radius: 4px; margin-top: 1rem;
}
.skeleton-overview.short { width: 80%; }

.skeleton-button {
  width: 150px; height: 45px; border-radius: 25px; margin-top: 1rem;
}
```

## Paso 4: Integrar los Skeletons en la Home

Ahora vamos a implementar estos componentes en nuestra página principal (Home).

**Nota Importante:** @defer sirve para la carga diferida del *código* de un componente. Para esperar a que lleguen los *datos de una API*, la forma más limpia en Angular moderno es usar el bloque @if verificando si los datos ya existen.

Abre **src/app/features/home/home.ts** e importa los dos componentes nuevos que creamos:

```
import { SkeletonHero } from '../../shared/components/skeleton-hero/skeleton-hero';
import { SkeletonCard } from '../../shared/components/skeleton-card/skeleton-card';
// Añádelos al arreglo 'imports: [...]' del decorador @Component.
```

Abre **src/app/features/home/home.html** y actualiza la lógica para mostrar los Skeletons si la información de TMDB aún no ha llegado:

### 4.1 El Hero

```
@if (trendingMovies().length > 0) {
  <!-- Aquí va tu componente <app-hero> normal con la película destacada -->
  <app-hero [movie]="trendingMovies()[0]"></app-hero>
} @else {
  <!-- Mientras está vacío (cargando), mostramos el esqueleto -->
  <app-skeleton-hero></app-skeleton-hero>
}
```

### 4.2 Actualizar el componente MovieSlider

Para los sliders, la forma más limpia es hacer que el propio MovieSlider maneje sus propios esqueletos cuando no recibe películas.

Abre **src/app/shared/components/movie-slider/movie-slider.ts** e importa el SkeletonCard:

```
import { SkeletonCard } from '../../skeleton-card/skeleton-card';
// Añádelo a los imports: [CommonModule, MovieCard, SkeletonCard]
```

Abre **src/app/shared/components/movie-slider/movie-slider.html** y modifica la pista del slider:

```
<div class="slider-track">
  @if (movies.length > 0) {
    @for (movie of movies; track movie.id) {
      <app-movie-card [movie]="movie"></app-movie-card>
    }
  } @else {
    <!-- Mostramos 5 esqueletos mientras carga -->
    @for (dummy of [1,2,3,4,5]; track dummy) {
      <app-skeleton-card></app-skeleton-card>
    }
  }
</div>
```

### 4.3 El Catálogo Infinito en la Home

Finalmente, abre **src/app/features/home/home.html** y actualiza la grilla del catálogo infinito:

```
<!-- Catálogo Infinito -->
<section class="catalog-section">
  <h2 class="section-title">Explorar Catálogo</h2>

  <div class="movies-grid">
    @if (catalogMovies().length > 0) {
      @for (movie of catalogMovies(); track movie.id) {
        <app-movie-card [movie]="movie"></app-movie-card>
      }
    } @else {
      <!-- Mostramos 10 esqueletos de tarjeta para rellenar el espacio visualmente -->
      @for (dummy of [1,2,3,4,5,6,7,8,9,10]; track dummy) {
        <app-skeleton-card></app-skeleton-card>
      }
    }
  </div>
```

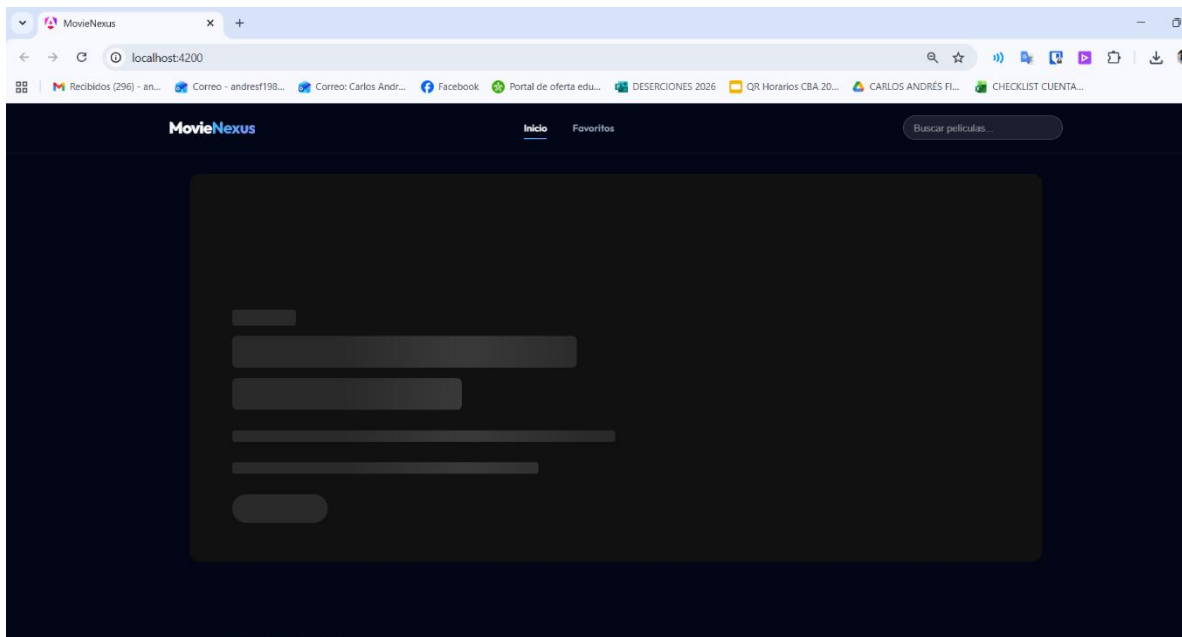
### Paso 5: Probar la Animación Artificialmente

Nuestra aplicación es tan rápida que quizás ni alcances a ver los Skeletons. Para apreciarlos, puedes simular una red lenta en tu navegador o poner un pequeño retraso (delay) temporal en tu servicio.

Abre **src/app/core/services/movie.service.ts** y usa delay de RxJS temporalmente para probar:

```
import { delay } from 'rxjs/operators'; // Importación nueva

// En alguna de tus peticiones:
getPopularMovies() {
  return this.http.get(...).pipe(
    delay(2000) // Retrasa la respuesta 2 segundos a propósito (¡Solo para probar!)
  );
}
```



## Prueba de Aprendizaje (Checklist)

1. **¿Cuál es la diferencia de UX entre usar un Spinner y un Skeleton Loader?** *(Respuesta: El spinner requiere que la pantalla se vuelva a calcular cuando llegan los datos (layout shift), mientras que el skeleton bloquea exactamente el espacio que usará el componente final, haciendo la transición casi invisible).*
2. **¿Por qué pusimos la clase .skeleton-base en el archivo de estilos global (styles.css)?** *(Respuesta: Porque el efecto de degradado y animación de brillo es exactamente el mismo para todos los esqueletos (tarjetas, botones, banners). Al hacerlo global, no repetimos código CSS y nos aseguramos de que el brillo esté sincronizado).*

## Control de Versiones

Has mejorado significativamente la percepción de velocidad de tu app. Guarda tus cambios:

```
git add .  
git commit -m "feat: implementar skeleton loaders para la home mejorando la UX y CLS"  
git push origin main
```